

FPGA-Accelerated Model Predictive Control

A Fixed-Point Finite-Control-Set Path-Tracking Controller on the Zynq xc7z007s

Design, Implementation and Hardware-in-the-Loop Validation

Item	Value
Target device	Xilinx Zynq xc7z007s-clg400-1 (Digilent Cora Z7-07S)
Programmable-logic clock	50 MHz (FCLK_CLK0)
Toolchain	MATLAB R2021b, HDL Coder 3.19, Vivado 2022.1, Vitis 2022.1
Controller revision	fcs_mpc_v4
Timing closure	WNS +10.449 ns, TNS 0.000, all constraints met
Logic utilisation	7,854 of 14,400 LUTs (54.5%)
Closed-loop result on hardware	2.54 laps, 10.20 m/s peak, 0.866 m peak tracking error
Repository	github.com/subdiduntil2/FPGA-Accelerated-Model-Predictive-Control

Revision 8 | 29 July 2026

Contents

1. Scope and application
2. Control formulation
3. From 136 candidates to 25: the separability restructuring
4. Hardware architecture and implementation results
5. Verification methodology
6. Results
7. Comparison against software and against related hardware work
8. Reproduction procedure
9. Planned work
- A. Appendix: register map
- B. Appendix: pitfalls encountered and their resolutions
- . References

Summary of the work

This report documents a finite-control-set model predictive controller (FCS-MPC) for vehicle path tracking, implemented in fixed-point logic on a low-cost Zynq device and validated against a bit-exact MATLAB reference at five independent levels. The controller solves its optimisation by enumeration rather than by a quadratic program, which makes it a natural fit for programmable logic: every candidate is evaluated in parallel combinational logic and the argument of the minimum is selected by a comparison tree.

Three results define the current state of the design. First, the controller closes timing on the target device with more than half of the clock period to spare, after a restructuring that reduced logic use by 54% and removed the controller from the critical path. Second, the loop was closed on hardware: the vehicle plant runs on the processing system, the controller runs in the programmable logic, and errors compound from step to step rather than being reset by recorded data. Third, the hardware and the reference model agree exactly on every integer quantity across all 801 samples of the run, with the only measurable difference traced to the print resolution of the on-board formatter.

The report also states the application boundary of the present configuration and the route to lifting it. The controller currently assumes a surveyed circuit with a precomputed speed profile. Section 9 describes the planned unknown-track configuration, which retains the same programmable-logic core.

1. Scope and application

1.1 What the present configuration assumes

The controller as delivered targets a **known circuit**. Two quantities are computed offline in MATLAB and loaded into the system before the run begins:

1. **The reference path.** A closed circuit is sampled into 358 vertices at approximately 0.5 m spacing and exported as the arrays REF_X and REF_Y in reference_track.h. The circuit combines two straights, two semicircular ends of 15 m radius, and a sinusoidal perturbation of 4 m amplitude on the upper straight, so that the controller is exercised on constant-radius corners and on continuously varying curvature within a single lap.
2. **The reference speed profile.** A speed is precomputed for every vertex from the local path curvature, subject to a lateral acceleration limit and to forward and backward acceleration passes, and exported as the int16 array VREF in the same header. Section 2.4 gives the construction.

During the run the processing system locates the nearest path vertex, walks a speed-dependent look-ahead distance along the path to obtain the position reference, reads the profile speed at the current vertex, quantises all six quantities, and hands them to the controller. The controller therefore never builds a map, never estimates curvature, and never computes a speed profile online. It consumes references that were prepared in advance.

This is the correct assumption for a surveyed test track or a race circuit, where the geometry is known before the vehicle drives it and the limiting speed at every point can be established from the geometry and the tyre limit. It is not the correct assumption for an unmapped road. Section 9 sets out the planned configuration that removes the assumption without changing the programmable-logic core.

1.2 System partition

The system splits across the two halves of the Zynq device. The programmable logic holds the MPC core, wrapped as an AXI4-Lite peripheral. The processing system holds everything else: the vehicle plant during validation, the nearest-point and look-ahead search, the fixed-point quantisation, the reference tables, and the logging. The two communicate through a memory-mapped register file at base address 0x43C0_0000.

The partition follows from what each side does well. The controller is a wide, branch-free computation over a fixed candidate set, so it maps onto parallel combinational logic and completes in a fixed number of clock cycles. The search, the bookkeeping and the plant integration are sequential and floating-point, so they belong on the processor. In a deployment the plant would be replaced by the vehicle and its sensors, and the same partition would hold.

1.3 Repository contents

The repository holds the current revision of every artifact described in this report. The MATLAB testbench is the single generator: the vector files, the C header and the golden trajectory all come from one run, which is what prevents the four implementations from drifting apart through independent recomputation.

File	Role
matlab/fcs_mpc_v4.m	The controller, and the specification against which every other implementation is checked
matlab/fcs_mpc_v4_tb.m	Closed-loop golden testbench. Builds the circuit and the speed profile, runs the loop, and writes every downstream artifact
matlab/fcs_mpc_v4_hdltb.m	Replays the nine exported vector files through the controller. Gates hardware generation
matlab/compare_fpga_matlab.m	Closed-loop comparison of the board capture against the golden run, 15 columns
hdl/fcs_mpc_v4.vhd	Generated combinational core, seven int16 inputs and two int16 outputs, with registered input and output stages
hdl/fcs_mpc_v4_pkg.vhd	Signed vector types used by the core
hdl/fcs_mpc_v4_tb.vhd	Generated self-checking testbench. Drives the vector files and raises testFailure on any command mismatch
hdl/fcs_mpc_v4_tb_pkg.vhd	Testbench support package
hdl/MPC_controller_AXI_v1_0.vhd	IP top level
hdl/ MPC_controller_AXI_v1_0_S00_AXI. vhd	Register file, kick, run and capture state machine, and the core instantiation
hdl/design_1*.vhd	Block design output and its wrapper
hw/design_1.bd	Block design source
hw/design_1_wrapper.xsa	Hardware handoff for Vitis, exported with the bitstream
hw/mpc_timing.xdc	Multicycle exception on the controller path, six cycles setup and five hold
src/mpc_closed_loop.c	Closed-loop harness with the software and FPGA backends and the deterministic plant
src/reference_track.h	Generated: the 358-vertex path, the VREF speed table, the plant constants and the scale factors
docs/	This report and the reference paper

Two files are absent by design. The controller is no longer hand-written, so there is no maintained VHDL source separate from the generated core; the MATLAB function is edited and the hardware is regenerated. And the open-loop replay application has been retired in favour of the closed-loop harness, which subsumes it, so the latency instrumentation it carried is not currently in the tree. Section 6.4 states the consequence for the reported latency figures.

2. Control formulation

2.1 Vehicle model and sampling

The prediction model is the kinematic bicycle model with a wheelbase L of 2.5 m, integrated at a sampling period T_s of 0.1 s. Writing the state as position (x, y) , heading ψ and speed v , and the inputs as steering angle δ and acceleration command a , the plant used for validation advances as

$$\begin{aligned} x(k+1) &= x(k) + v(k) \cos(\psi(k)) T_s \\ y(k+1) &= y(k) + v(k) \sin(\psi(k)) T_s \\ \psi(k+1) &= \psi(k) + (v(k)/L) \tan(\delta(k)) T_s \\ v(k+1) &= (v(k) + a(k)) * 0.99 \end{aligned}$$

The factor 0.99 is a lumped drag term applied once per step. The controller carries a simplified form of the same model internally: it predicts the next heading and the next position for each steering candidate, and the next speed for each acceleration candidate, one step ahead. The prediction horizon is exactly one step. A longer horizon would multiply the candidate count by itself and is not affordable in the target device at this candidate density.

2.2 Finite control set

The actuation is continuous in the physical vehicle, so the candidate set is a design choice rather than a property of the plant. Steering is discretised into 17 options and acceleration into 8, both in controller units:

```
STEER_OPTS = [0, -2, 2, -4, 4, -7, 7, -10, 10, -14, 14, -18, 18, -22, 22, -26, 26]
ACCEL_OPTS = [0, 20, 10, 4, -4, -10, -20, -31]
```

One steering unit decodes to 0.025 rad, so the set spans plus or minus 0.65 rad, about 37 degrees. The spacing is deliberately non-uniform: it is dense near zero, where small corrections dominate on straights and in shallow curvature, and coarse at the extremes, where only the sign and rough magnitude matter. The acceleration set is asymmetric, with a stronger maximum deceleration (-31) than acceleration (+20), because braking into a corner is the operation that limits how aggressively the speed profile can be followed.

Hard limits are enforced by construction rather than by constraint handling. The steering and acceleration bounds are the extremes of the candidate lists, so an infeasible input cannot be produced. The speed bound is enforced by saturating the predicted speed at $V_{MAX} = 640$ units, which corresponds to 10 m/s. There is no active-set logic and no projection step.

2.3 Cost function

For a steering candidate s and an acceleration candidate a , the cost is

$$\begin{aligned} J(s, a) = & W_ERR * (\text{clamp}(|ex(s)|) + \text{clamp}(|ey(s)|)) && \text{tracking} \\ & + W_STEER * |s| && \text{steering effort} \\ & + W_OVER * \min(\max(v_next(a) - v_ref, 0), OV_MAX) && \text{overspeed penalty} \\ & - W_SPEED * \min(v_next(a), v_ref) && \text{capped speed reward} \end{aligned}$$

$W_ERR = 15 \quad W_STEER = 2 \quad W_SPEED = 2 \quad W_OVER = 6 \quad OV_MAX = 255 \quad \text{clamp at } 1000$

The tracking term is a clamped L1 norm of the predicted position error, not a squared norm. With many candidates evaluated in parallel, one squaring multiplier per candidate is expensive, and the L1 form gives the same ordering behaviour for the purpose of selecting an argument of the minimum. The clamp at 1000 units, roughly 15 m per axis, prevents a large excursion from saturating the int16 accumulator and keeps the effort terms meaningful when the vehicle is far off the path.

The speed terms are the pair that distinguish this revision from earlier ones. The reward for speed is capped at the profile value, so the controller gains nothing by exceeding the reference, and any excess is charged at a rate

three times the reward gradient through W_{OVER} . The result is a controller that rides the profile rather than pushing against a single global speed limit, which is what makes the 10 m/s ceiling usable on a circuit whose corners demand 2.5 m/s.

Ties are broken by strict inequality and first occurrence. A candidate replaces the incumbent only if its cost is strictly lower, so the earliest candidate in list order wins among equals. This rule is not cosmetic: it is the property that has to be reproduced identically in MATLAB, in C and in VHDL for the three implementations to agree bit for bit, and it is why the candidate lists are ordered as they are rather than sorted.

2.4 Reference speed profile

The profile generator runs once, offline, inside the MATLAB testbench. It proceeds in four stages over the closed path:

1. **Curvature.** The Menger curvature is evaluated at every vertex from the triangle formed with the vertices two positions before and after, using twice the triangle area divided by the product of the three side lengths.
2. **Windowed maximum.** Each vertex takes the maximum curvature over a window of plus or minus three vertices. This is deliberately conservative and prevents the profile from admitting a speed that the vehicle cannot hold through the tightest part of a corner it is about to enter.
3. **Lateral limit.** The admissible speed is the smaller of $V_{\text{TOP}} = 10$ m/s and the square root of A_{LAT} divided by the curvature, with $A_{\text{LAT}} = 4$ m/s².
4. **Feasibility passes.** Two backward passes enforce that the vehicle can decelerate into each corner at $A_{\text{BRK}} = 3.5$ m/s², and two forward passes enforce that it can only accelerate out at $A_{\text{ACC}} = 3$ m/s². The passes wrap around the closed circuit, so the profile is consistent lap to lap.

The result is scaled by 64, rounded to int16, saturated at 640, and written into `reference_track.h`. The same table is read by the MATLAB testbench and by the C harness, so neither side can drift from the other through an independent recomputation. This shared-artifact discipline is applied throughout and is what makes the cross-implementation comparisons meaningful.

2.5 Fixed-point representation

There is no floating-point arithmetic anywhere on chip. Every controller signal is a small signed integer with a scale factor folded into the constants and the shift counts.

Quantity	Scale	Resolution	Notes
Position x, y	64 units per metre	15.6 mm	Q10.6 over the circuit extent
Heading psi	41 units per radian	0.0244 rad (1.4 deg)	One unit is one entry of the 256-point trigonometric table
Speed v	64 units per m/s	15.6 mm/s	Saturated at 640 (10 m/s)
Steering command	0.025 rad per unit	1.43 deg	Signed 6-bit payload
Acceleration command	1/64 m/s per unit	15.6 mm/s per step	Signed 6-bit payload

The heading scale is chosen so that a full turn maps onto 256 units, which makes the trigonometric lookup a bitwise AND with 255 rather than a modulo operation. The sine and cosine tables are 256-entry, 8-bit, and built by folding a single 64-entry quadrant, so the constant storage is one quarter of the apparent size. The kinematic update uses only shifts:

```
dpsi = (v * delta) >> 9
dx   = ((v >> 4) * cos_lut[psi_next & 255]) >> 5
dy   = ((v >> 4) * sin_lut[psi_next & 255]) >> 5
```

The shift counts are pre-multiplied so that the position increments emerge directly in the position scale. The heading shift was widened from 8 to 9 places relative to the earlier 5 m/s revision. The previous value overestimated the heading change that a given steering input produces at speed, so the controller selected too little steering and understeered in mid-corner above about 6.5 m/s.

3. From 136 candidates to 25: the separability restructuring

3.1 The problem

The direct statement of the controller evaluates the full product set: for each of the 17 steering options, and for each of the 8 acceleration options, predict the next state, evaluate the cost, and compare against the running minimum. That is 136 candidates. Synthesised as parallel combinational logic, it requires 136 copies of the kinematic prediction, which is the expensive part of the design, since each copy contains a multiply, a table lookup, two more multiplies, two shifts and two absolute values.

The measured consequence was decisive. Synthesis reported 16,939 look-up tables against the 14,400 available on the xc7z007s, that is 118% utilisation. Implementation did not fail on timing; it failed before the placer started, because the design did not fit. Neither a constraint change nor a synthesis directive addresses a shortfall of that size.

3.2 The observation

Inspection of the cost function shows that the two decision variables never meet. Returning to Section 2.3, the tracking and steering-effort terms depend only on the steering candidate, and the overspeed and reward terms depend only on the acceleration candidate. The cost is therefore separable:

$$J(s, a) = A(s) + B(a)$$

$$A(s) = W_ERR * (clamp|ex(s)| + clamp|ey(s)|) + W_STEER * |s|$$

$$B(a) = W_OVER * overspeed(a) - W_SPEED * reward(a)$$

The separation is exact, not an approximation. It holds because the one-step position prediction ignores the effect of the acceleration candidate on the distance travelled within the step, a property of the model as specified that was already true of the earlier revision and had simply not been exploited.

Separability decomposes the minimisation as well as the evaluation. Under the outer-steering, inner-acceleration scan order of the original code, the first candidate achieving the minimum cost is exactly the pair formed by the first steering option minimising A and the first acceleration option minimising B. The tie-breaking rule survives the decomposition, which is the point that had to be established before the restructuring could be called equivalent rather than merely similar.

3.3 Effect

The restructured controller evaluates 17 kinematic cones and 8 speed terms, and performs two independent first-minimum scans of depth 17 and 8 in place of one scan of depth 136. Before regenerating any hardware, three checks were run: the int16 accumulators cannot saturate under either grouping, with a worst case of 31,582 against the 32,767 limit; the restructured function reproduces the original on 800 random states with zero mismatches; and it reproduces it on all 801 samples of the closed-loop run, giving identical lap count, peak error and top speed.

Quantity	Direct form (136 candidates)	Separated form (17 + 8)
Kinematic prediction instances	136	17
Argmin comparison depth	136	17 and 8, independent
Slice LUTs after synthesis	16,939 (118% of device)	7,854 (54.5% of device)
Implementation outcome	Placer not reached	Routed, all constraints met
Controller outputs	reference	bit-identical to reference

Logic use fell by 54%, and the critical path shortened by roughly the same factor. After implementation the worst setup path no longer passes through the controller at all; it belongs to the AXI reset and handshake logic feeding the control register. The restructuring therefore resolved the area failure and the timing pressure together, at no cost in behaviour.

4. Hardware architecture and implementation results

4.1 The controller core

The core is generated from the MATLAB source by HDL Coder as a purely combinational entity with seven 16-bit inputs and two 16-bit outputs, plus a single registered stage at the input and at the output. Generation is driven from the command line with explicit `int16` argument types, which bypasses the fixed-point conversion stage entirely. That choice is deliberate: the MATLAB source is already in fixed point with hand-chosen widths, and the conversion tool would propose its own quantisation and could silently alter the widths that the bit-exactness argument depends on.

The constants are emitted as VHDL arrays: the 17 steering options and 8 acceleration options as 8-bit signed vectors, and the two trigonometric tables as 256-entry 8-bit signed vectors. No block RAM is inferred; the tables synthesise into logic, which is why the design uses no BRAM at all.

4.2 AXI4-Lite peripheral and handshake

The core is wrapped in a custom AXI4-Lite slave, `MPC_controller_AXI`, mapped at `0x43C0_0000` with a 64 kB aperture. The register file exposes seven input words, two output words, a control word, a status word and a transaction counter. Appendix A gives the full map. The processing system drives one solve as follows:

A three-state machine inside the wrapper sequences the solve. On the kick write it leaves idle, counts a fixed number of cycles while the combinational cloud settles, then captures the two outputs into the result registers, sets the done flag and increments the transaction counter. The input registers carry a `DONT_TOUCH` attribute so that synthesis cannot merge or retime them, which would invalidate the timing exception described next.

4.3 The multicycle constraint

The only register-to-register path through the controller runs from the seven input registers, through the cost cloud, to the two output registers. The default analysis assumes that this path must settle within one clock period, that is 20 ns at 50 MHz. That requirement is wrong for this design. The capture edge occurs six clocks after the kick write, and the input registers cannot change on the kick cycle, because that write decodes to the control address. The data therefore has six periods, 120 ns, to settle.

The constraint file states this relationship explicitly:

```
set_multicycle_path 6 -setup \  
-from [get_cells -hier -regexp {.*MPC_S00_AXI_inst/slv_reg(4|5|6|7|8|9|10)_reg.*}] \  
-to   [get_cells -hier -regexp {.*MPC_S00_AXI_inst/slv_reg(12|13)_reg.*}]  
  
set_multicycle_path 5 -hold \  
-from [get_cells -hier -regexp {.*MPC_S00_AXI_inst/slv_reg(4|5|6|7|8|9|10)_reg.*}] \  
-to   [get_cells -hier -regexp {.*MPC_S00_AXI_inst/slv_reg(12|13)_reg.*}]
```

The setup and hold values are coupled, N and N minus one, and both are tied to the state machine terminal count through the relation `wait_cnt = N minus 2`. The present values are $N = 6$ and `wait_cnt = 4`. Raising the budget therefore means editing both files together; each additional cycle buys 20 ns of settling time and costs 20 ns of solve latency, which is negligible against a 100 ms sampling period. Note that XDC files accept only a restricted subset of Tcl, so conditionals, loops and expression evaluation are rejected by the parser.

The routed timing summary does not resolve whether the exception is what makes the path close, because it lists only the ten worst setup paths and none of them terminates at the result registers. What it does establish is a lower bound: the controller path has at least 10.449 ns of setup slack under whichever requirement was applied to it, and all 2,077 setup and 2,077 hold endpoints in the design meet their requirements. Retiring the constraint therefore requires a targeted query on the implemented design: report the path from the input registers to the result registers and read whether the requirement is 20 ns or 120 ns. Until that is done the constraint should be retained, since a design that closes only because of an exception will fail silently if the exception is removed.

4.4 Implementation results

The results below are taken from the routed design, Vivado 2022.1, part xc7z007s-clg400-1, 50 MHz.

Timing metric	Value	Status
Worst negative slack (WNS)	+10.449 ns	Met, over half the 20 ns period spare
Total negative slack (TNS)	0.000 ns, 0 failing endpoints	Met
Worst hold slack (WHS)	+0.040 ns	Met
Total hold slack (THS)	0.000 ns	Met
Worst pulse width slack (WPWS)	+9.020 ns, 0 of 935 failing	Met
Endpoints analysed	2,077 setup, 2,077 hold	All met
Unconstrained paths	none	Fully constrained

The report states that all user-specified timing constraints are met, with no unconstrained paths and a single clock domain at 20 ns. The worst setup path does not run through the controller. It runs from the processor system reset synchroniser to the reset pin of the control register:

```
Source:      design_1_i/rst_ps7_0_50M/U0/ACTIVE_LOW_PR_OUT_DFF[0].FDRE_PER_N/C
Destination: design_1_i/MPC_controller_AXI_0/U0/MPC_S00_AXI_inst/slv_reg0_reg[1]/R
Requirement: 20.000 ns      Data path delay: 8.580 ns
Logic levels: 1 (LUT1)      logic 0.580 ns (6.8%), route 8.000 ns (93.2%)
```

One logic level and 93% route delay identify this as a high-fanout reset distribution path rather than a computation path. The controller has ceased to be the timing bottleneck, and the limiting path is now a property of the block design infrastructure. Taken on this path alone, the design would close at roughly 105 MHz.

Resource	Used	Available	Utilisation
Slice LUTs	7,854	14,400	54.54%
LUT as logic	7,794	14,400	54.13%
LUT as shift register	60	6,000	1.00%
Occupied slices	2,361	4,400	53.66%
Slice registers (all flip-flops)	870	28,800	3.02%
F7 multiplexers	374	8,800	4.25%
F8 multiplexers	165	4,400	3.75%
CARRY4 chains	1,243	4,400	28.25%
DSP slices	40	66	60.61%
Block RAM tiles	0	50	0%
Global clock buffers (BUFG)	1	32	3.13%
MMCM / PLL	0	2 / 2	0%
Unique control sets	38	4,400	0.86%

The primitive census shows where the logic goes:

Primitive	Count	Function
LUT3 / LUT4	2,658 / 2,524	Dominant pair: the clamped absolute-value and compare-select logic
LUT6	2,144	The 256-entry trigonometric tables, read at 17 addresses at once
LUT5 / LUT2 / LUT1	1,557 / 1,014 / 194	Remaining combinational glue
CARRY4	1,243	Adder and comparator chains in the cost accumulation and the argmin
MUXF7 / MUXF8	374 / 165	Wide multiplexing in the table lookups and the candidate selection

Primitive	Count	Function
DSP48E1	40	Products in the 17 kinematic cones
FDRE / FDSE	849 / 21	All 870 flip-flops carry a clock enable; 849 synchronous reset, 21 synchronous set
SRLC32E / SRL16E	45 / 19	AXI interconnect pipelining, mapped to the 60 memory LUTs
BUFG / PS7 / BIBUF	1 / 1 / 130	Single clock buffer, the processing system, and its fixed I/O pads

The 10,091 LUT primitives occupy only 7,794 logic sites because 2,297 of them use both the O5 and O6 outputs of a single six-input LUT. That packing is what keeps a design of this width inside a device this small.

The DSP count is the one figure that can be attributed to the controller with confidence. Neither the processing system, which is hard silicon, nor the reset synchroniser, nor the AXI protocol converter infers a DSP48E1, so all 40 belong to the MPC core. That is 2.35 DSPs per kinematic cone, consistent with the two products in the position update and the shared product in the heading update.

The profile is characteristic of the algorithm. Logic dominates, because the design is a wide combinational evaluation with very little state; the register count of 870 covers little more than the AXI register file and the state machine. The 40 DSP slices absorb the products in the 17 kinematic cones. No block RAM is used, since the trigonometric tables are small enough to synthesise into logic and are read at 17 addresses simultaneously, which a block RAM could not serve without replication. Roughly 6,500 LUTs remain free for future work.

4.5 Power

Power was estimated on the routed design with the Vivado analyser. The estimate is vectorless: no switching activity file was supplied, so the tool applies default activity rates and reports its own confidence as medium. The figures are therefore indicative of the distribution between blocks rather than exact absolute values, and the distribution is the interesting result.

Component	Power (W)	Share of total
Processing system (PS7)	1.251	90.6%
MPC peripheral, core and AXI slave	0.012	0.9%
AXI interconnect	0.002	0.1%
Clocks, signals and remaining logic	0.001	0.1%
Total dynamic	1.266	91.7%
Device static	0.115	8.3%
Total on-chip	1.381	100%

The complete programmable-logic side of the design draws about 14 mW, of which the MPC peripheral accounts for 12 mW. The processing system draws 1.251 W, roughly a hundred times more. That ratio is the practical argument for the accelerator, and it is stronger than the timing argument. The processor is present in any case and its power is largely fixed, so moving the control law into the fabric adds about 12 mW to a system that already draws over 1.2 W, in exchange for the speed-up quantified in Section 7.1. Running the same algorithm in software on the processor would not avoid that 1.251 W; it would occupy it.

Thermal margin is ample. The junction temperature is estimated at 40.9 C with an effective junction-to-ambient resistance of 11.5 C/W, giving a maximum ambient of 69.1 C. No active cooling is required, which is consistent with the low-cost target platform.

The clocking is minimal, which contributes to the low fabric power. The design runs on one clock domain sourced from the processor at 50 MHz, distributed through a single global buffer driving 935 loads, with no mixed-mode clock manager and no phase-locked loop instantiated.

5. Verification methodology

5.1 Five levels

The project treats the MATLAB function as the specification and every other implementation as a translation of it. The claim to be established is not that the hardware behaves reasonably, but that it produces the same integers as the specification for the same inputs. Five checks build that claim, each one removing a different way in which the previous one could be satisfied by a design that is nevertheless wrong.

Level	What is compared	What it rules out	Result
1. Algorithmic equivalence	Restructured controller against the previous revision, in MATLAB	The separability argument being wrong	0 mismatches on 801 closed-loop samples and 800 random states
2. Golden-vector replay	MATLAB controller against the exported .dat vectors	The exported vectors not describing the run	801 / 801 match
3. RTL simulation	Generated VHDL against the same vectors, in Vivado xsim	The generated hardware differing from the source	801 / 801 pass, both failure flags clear
4. Software closed loop	C harness against the MATLAB golden run, on a host	The C port or the shared plant differing	0 integer mismatches, 0 m position difference
5. Hardware closed loop	Cora running the loop against the MATLAB golden run	Everything downstream of simulation	0 integer mismatches, 7.07e-7 m position difference

Levels 1 to 3 are open-loop: they present recorded inputs and check the outputs. That is necessary but not sufficient, because each sample starts from a known-good state and an error cannot propagate. Levels 4 and 5 close the loop, so any disagreement compounds instead of resetting. A single incorrect low-order bit would separate the two trajectories visibly within one lap.

5.2 Making two floating-point implementations agree exactly

Closing the loop introduces a problem the open-loop replay never had. Both sides now integrate the plant, so every double-precision operation must produce the identical bit pattern in MATLAB on a workstation and in C on a bare-metal Cortex-A9. Library implementations of sine, cosine, square root and rounding do not guarantee that. The harness therefore links no math library on the target and replaces each sensitive primitive:

- **Trigonometry.** A shared routine reduces the argument to the interval from minus pi to pi using a deterministic round, folds the quadrant, and evaluates fixed polynomials of degree 13 for sine and 14 for cosine with hard-coded coefficients. Only IEEE-754 additions and multiplications appear, in a fixed order. MATLAB evaluates the identical polynomial.
- **Square root.** The ARM floating-point instruction VSQRT.F64 is emitted through inline assembly. It is correctly rounded by the standard, so it matches the host and MATLAB exactly.
- **Rounding and quantisation.** Floor, round and the int16 conversion are built from integer casts that reproduce round-half-away-from-zero with saturation, matching the MATLAB semantics without a library call.

One compiler flag is mandatory rather than advisory. The application must be built with `-ffp-contract=off`, otherwise the compiler fuses multiply-add pairs in the plant integration into single fused instructions with a different rounding behaviour, and reintroduces exactly the divergence the deterministic primitives were written to remove.

5.3 Guaranteeing that the hardware produced the result

A closed-loop harness that can compute the control law in software is a harness that might do so. The guarantee here is structural rather than procedural: the software controller, the two trigonometric tables and the table constructor are all inside a conditional compilation block that is excluded when the FPGA backend is selected. The binary loaded onto the board does not contain a software path to a command, so the only possible source is the AXI read. This was confirmed by inspecting the symbol table of the compiled object.

The register interface is checked rather than assumed: the controller inputs are read back from the peripheral and compared against what the processor wrote, so a truncation or a mis-decoded address appears as a mismatch instead of passing silently.

6. Results

6.1 Reference behaviour before hardware

Figure 1 shows the closed-loop reference run: 801 steps at 0.1 s, that is 80 s of simulated driving, with the controller in the loop and the plant integrated in double precision. The panels are, from top to bottom, the trajectory against the circuit, the vehicle speed against the reference profile, the steering command, the acceleration command, and the distance to the nearest path vertex.

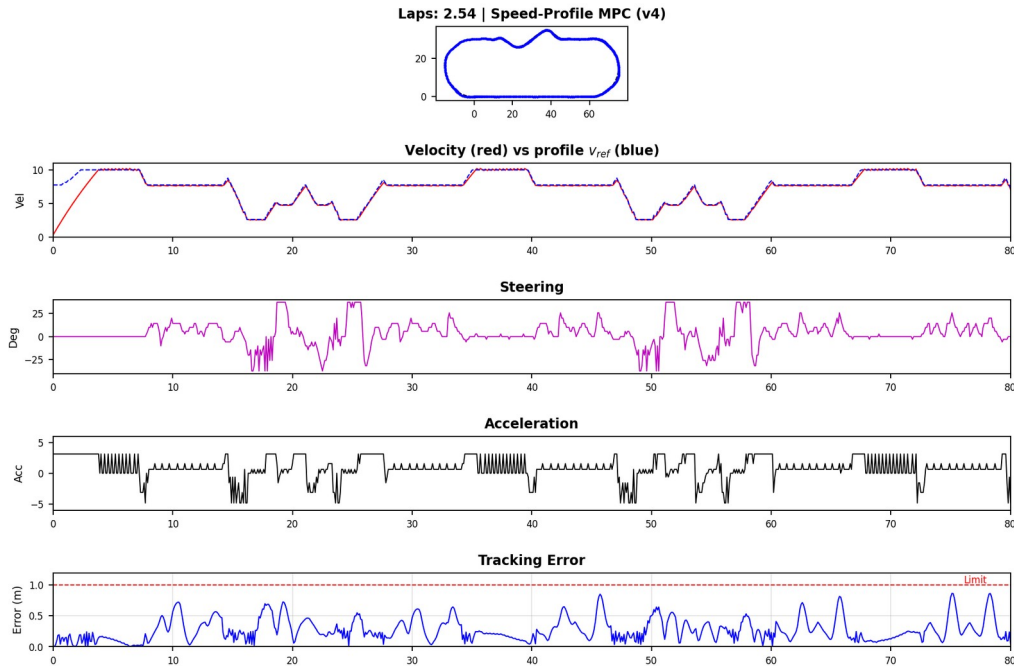


Figure 1. Reference closed-loop run before hardware deployment. The controller completes 2.54 laps in 80 s, reaches 10.2016 m/s, and holds peak tracking error at 0.866 m against the 1.0 m acceptance limit.

Three features are worth reading off the figure. The speed trace follows the profile rather than saturating at a single value: it holds 10 m/s on the straights, drops to about 2.5 m/s in the tight sections, and shows the intermediate plateaus the sinusoidal segment demands. The acceleration trace switches densely between adjacent candidates on the plateaus, the expected behaviour of a finite control set holding a setpoint it cannot represent exactly; the amplitude is one candidate step and does not appear in the speed trace at this scale. Tracking error stays below the 1.0 m limit throughout. Its peaks fall at corner entry, where the look-ahead reference moves fastest relative to the vehicle.

6.2 Agreement between hardware and reference

The same loop was then run on the board with the plant on the processing system and the controller in the programmable logic. The 801-row capture was compared column by column against the reference run.

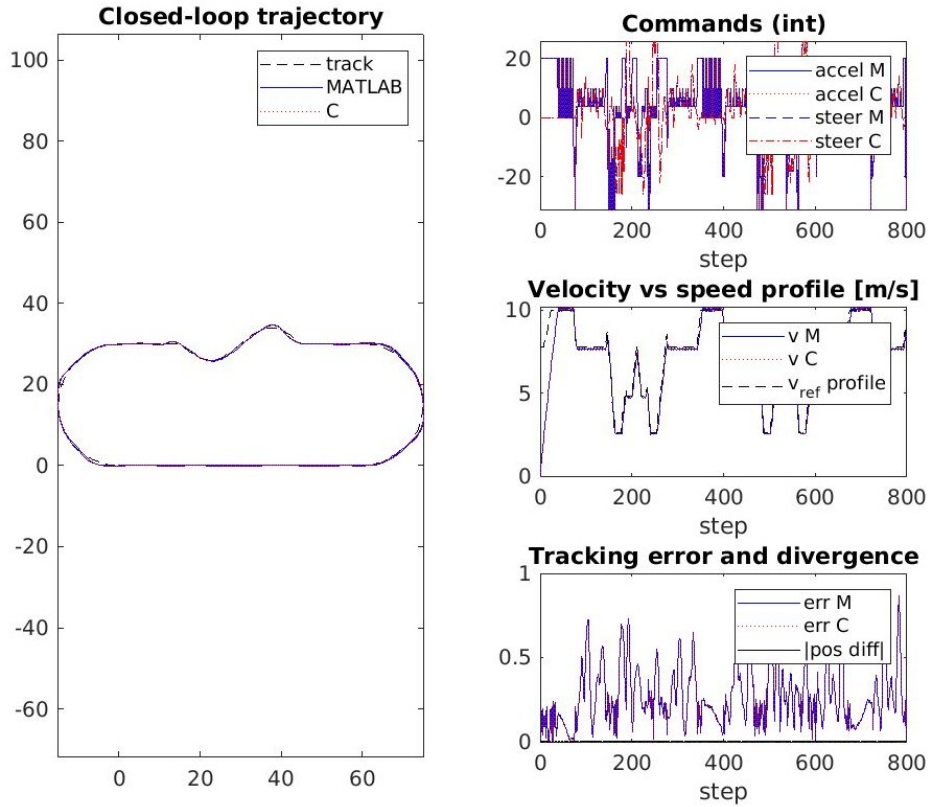


Figure 2. Closed-loop validation, produced by `compare_fpga_matlab.m` from the MATLAB golden run and the Cora UART capture (labelled C). Left: the circuit with both trajectories overlaid. Top right: the integer command sequences from both sides. Centre right: vehicle speed from both sides against the reference profile. Bottom right: per-step tracking error of each side and the absolute distance between the two trajectories, which stays on the axis floor for the whole run.

Comparison	Result
Rows compared	801
Integer-exact rows (quantised state, references, commands)	801 of 801
Acceleration command differences	0
Steering command differences	0
Reference-profile column differences	0
Maximum position difference	7.07e-7 m (mean 3.64e-7 m)
Top speed, reference and hardware	10.2016 m/s on both sides
Peak tracking error, reference and hardware	0.8664 m on both sides
Lap count, reference and hardware	2.54 on both sides

The capture was also re-checked independently of the comparison script, by replaying it against a separate reimplementation of the controller and the plant. That check agrees: 801 rows, zero mismatches across all nine integer columns, mean position difference 3.64e-7 m and maximum 7.0e-7 m, with the same top speed and peak error. The result is therefore not an artefact of the comparison script.

Every quantity that the controller consumes or produces is identical. The quantised state, the quantised references, the profile speed and both returned commands agree at every one of the 801 steps, so the hardware and the specification made the same decision at every step of the run.

6.3 Interpreting the residual difference

The remaining 7.07e-7 m appears only in the printed double-precision columns, and it is an artifact of formatting rather than of arithmetic. The bare-metal build has no floating-point printf, so the harness formats doubles by hand into six decimal places, while MATLAB writes with ten significant digits. Of 4,005 double comparisons, 3,999

agree exactly once the MATLAB value is rounded to six decimals. The six that do not all carry a digit 5 in the seventh decimal, where the hand-written formatter truncates and MATLAB rounds. The discrepancy is one unit in the last printed digit.

The decisive evidence that the underlying values are identical comes from the quantised columns, which are derived from those same doubles. Across 4,806 quantisation events, not one produced a different integer. Real drift of any magnitude would eventually place a value on the opposite side of a rounding boundary and flip an integer. None flipped. Widening the on-board formatter from six to nine fractional digits would drive the reported difference to zero, but it would not change any decision the controller made.

The expected divergence is exactly zero, by construction rather than by luck. The controller is integer-only and its equivalence was established independently in RTL simulation, and the plant uses deterministic primitives on both sides, so identical operations are performed in identical order on identical bit patterns.

6.4 Latency

Latency was measured on the present core, in the closed loop, on the board. The harness reads the Cortex-A9 global timer at four points inside the AXI sequence and emits two intervals per step as extra CSV columns. The programmable-logic interval runs from the kick write to the cycle on which the processor observes the done flag. The end-to-end interval spans the whole per-step exchange: seven input writes, the kick, the polling, and the two result reads. Statistics over all 801 steps:

Interval	Minimum	Average	Maximum
Programmable logic, kick to done	335 ns	341 ns	495 ns
End to end, first write to last read	3,741 ns	3,762 ns	4,436 ns

The timer ticks at 3 ns and a timestamp read costs 43 ns, both reported by the run itself, so the instrument is far finer than the quantities measured. The instrumentation did not disturb the loop: the run still reports 2.541899 laps and 0.866388 m peak error, the same trajectory as the uninstrumented capture.

Both maxima belong to the first sample. Step 0 records 495 ns and 4,436 ns; every later step falls between 335 and 344 ns, and between 3,741 and about 3,809 ns. The first transaction pays for cold caches, an untrained branch predictor and the first traffic through the interconnect. Excluding it, the programmable-logic interval varies by 9 ns across 800 steps, which is three timer ticks. The path is deterministic to the resolution of the instrument.

The 341 ns average decomposes cleanly, and the decomposition is the useful result:

Contribution	Value	Source
Controller settling window	120 ns	Six clocks at 50 MHz, fixed by the capture state machine
AXI status read round trip	178 ns	Remainder, the processor cannot see the done flag any sooner
Timestamp read	43 ns	Measured by the run, the cost of the instrument itself
Measured total	341 ns	

Only 120 ns of the figure is the controller, and that is a design constant rather than a measurement. The rest is the cost of asking. End to end the same holds more strongly: subtracting the timer overhead and the poll window leaves 3,421 ns for eight posted writes and two reads, about 342 ns per AXI transaction, because the memory barrier in each write prevents them overlapping. Bus traffic is 96.8 percent of the per-step cost; the controller is 3.2 percent.

Against the 100 ms sampling period the whole exchange occupies 0.0038 percent of the budget, so there is room for a much denser candidate set, a longer horizon, or a far higher sampling rate before the controller becomes the constraint. The one effective optimisation is fewer bus transactions rather than a faster core.

One comparison needs care. The earlier replay application, running the previous core, reported 739.8 ns for the programmable-logic interval and 3,752.7 ns end to end. The end-to-end figures agree to 0.25 percent, as expected since the bus path did not change. The programmable-logic figures do not, and the difference cannot be a speed-up: the previous state machine waited five cycles where the present one waits six, so the current design holds its result 20 ns longer. The discrepancy is definitional, in what the earlier application placed inside its window. The figures here supersede it because the method is documented.

7. Comparison against software and against related hardware work

7.1 Against MCU, CPU and GPU implementations

The table gathers published latency and power figures for embedded MPC on the platform classes that compete with programmable logic. Power is marked by how it was obtained, because the distinction matters: (m) is measured on the running system, (d) is a device datasheet or thermal design figure, and (e) is the vectorless estimate of Section 4.5. Energy per step is the product of latency and power, and it is the fairest single number when platforms differ in both.

Platform and device	Latency	Power	Energy per step	Ref
MCU, Cortex-M4 at 168 MHz, ADMM QP	0.1 to 2 ms	0.2 W (d)	20 to 400 μ J	[4]
MCU, Cortex-M7 at 600 MHz, ADMM QP	0.1 to 1 ms	0.35 W (d)	35 to 350 μ J	[3]
DSP, six-phase drive, FCS-MPC, 64 states	67.4 μ s	0.5 to 1 W (d)	34 to 67 μ J	[5]
Automotive ECU, dSPACE MicroAutoBox III, NMPC	14 to 22 ms	not reported	not reported	[6]
Embedded CPU, six-core Cortex-A78, MPPI	17.37 ms	4.61 W (m)	80.16 mJ	[2]
Embedded GPU, Jetson Orin Nano, MPPI	7.24 ms	5.17 W (m)	37.44 mJ	[2]
FPGA, ZCU102 at 200 MHz, MPPI	2.33 ms	6.39 W (m)	14.90 mJ	[2]
FPGA, ARTY Z7-20, FCS-MPC converter	30.1 ns clock period	not reported	not reported	[1]
This work, xc7z007s, whole chip	3.76 μ s per step	1.381 W (e)	5.20 μ J	
This work, accelerator increment	120 ns core window	0.012 W (e)	45 nJ	

Three points come out of the table. Latency spans five orders of magnitude across the platform classes, and power spans three. The energy column compresses both and separates the platforms more cleanly than either alone: the microcontrollers are frugal but slow, the GPU is fast but expensive to run, and the programmable-logic entries win on the product. The GPU result deserves a second look, since it does not behave the way intuition suggests. Desai and colleagues [2] measure their FPGA drawing more average power than the Jetson it replaces, 6.39 W against 5.17 W, yet consuming less than half the energy, because it finishes in a third of the time. A GPU does not reduce latency for a small problem in any case. Kernel launch and transfer overheads alone exceed the entire per-step budget of this design, and GPUs earn their place by making the sample count large rather than by finishing sooner.

This design sits two to three orders of magnitude below every entry in the energy column. That result is real but it must be read alongside the last column of Section 7.2, because the problems are not the same size. The MPPI entries evaluate thousands of Monte Carlo rollouts across a forty-step horizon. The NMPC entries solve a nonlinear program with tyre models and obstacle constraints. This controller evaluates twenty-five terms at a horizon of one. Quoting a 2,800-fold energy advantage over [2] would be arithmetic rather than engineering, and it would discredit the comparisons that do hold.

The defensible claim is narrower and still useful. For horizon-one finite-control-set path tracking, this design reaches microsecond end-to-end latency for 12 mW of incremental power on a device costing under 150 euro, which places it in the latency and energy class of motor-drive FCS-MPC on a DSP [5] while addressing a vehicle-guidance task that normally requires the millisecond-class platforms in rows four to seven. The price is the horizon-one restriction, which those platforms do not carry. Two caveats travel with the numbers. The power figure for this work is a vectorless estimate against measured values elsewhere, although Section 4.5 argues the error points downward. And the datasheet power figures in rows one to three describe the device rather than the solve, so they overstate the energy of a well-managed microcontroller that idles between control steps.

One comparison stays inside the same algorithm and the same arithmetic. Running this controller in C on the Cortex-A9 beside the fabric would take an estimated 100 to 500 microseconds, a range consistent with published embedded solver benchmarks [7], against 120 ns in logic. It would also occupy the processor that draws 1.251 W rather than adding 12 mW alongside it. That is the comparison the design was built to win, and it does not depend on any of the published figures above.

7.2 Against Purraji, Zamiri and de Castro, 2025

Purraji, Zamiri and de Castro [1], included in the docs folder, implement an FCS-MPC for a grid-connected single-phase H-bridge converter, generate the VHDL automatically with Simulink HDL Coder, and validate it with a hardware-in-the-loop model and the integrated logic analyser on an ARTY Z7-20. The two controllers belong to the same family: one-step finite control set, enumeration over a discrete candidate set, cost evaluation per candidate, argument of the minimum, no quadratic program and no iterative solver. Almost everything else differs, and the differences are informative.

Aspect	Purraji et al. 2025	This work
Plant	Grid-connected single-phase H-bridge	Kinematic bicycle vehicle model
Controlled quantity	Inductor current injected to the grid	Vehicle position on a reference circuit
Candidate set	3 switching states, physical	17 x 8, a design choice discretising continuous actuation
Horizon	1 step	1 step
Prediction model	Forward-Euler scalar current update	Nonlinear kinematic update needing sine and cosine
Cost function	Squared tracking error	Clamped L1 position error, steering effort, capped speed reward, overspeed penalty
Arithmetic	Fixed point, 32-bit signals, DSP-mapped products	int16 signals, 6 to 16 bit intermediates, shift-based scaling
Trigonometry	Not required, linear circuit model	Two 256-entry quarter-folded 8-bit tables
Design route	Simulink model, VHDL from HDL Coder	MATLAB int16 source, VHDL from HDL Coder
Execution model	Whole algorithm in one clock period, registered I/O	Combinational evaluation, registered I/O, six-cycle kick-to-capture handshake at 50 MHz
Reported timing	Minimum clock period 30.125 ns, 500 ns experimental sampling step	120 ns core window, 341 ns measured solve, 20 ns clock period, 10.449 ns slack
Controller resources	216 LUTs, 7 FFs, 8 DSPs on ARTY Z7-20	40 DSPs, all attributable to the controller. LUT and FF figures are whole-design (7,854 / 870) and not separated
Controller power	Not reported	12 mW for the peripheral, against 1.251 W for the processing system
System integration	Standalone PL design	Memory-mapped AXI4-Lite peripheral under processor control
Verification	Simulink simulation, then HIL with logic-analyser waveforms	Bit-exact equivalence at five levels, including a closed loop on hardware

The candidate set is physical in one case and synthetic in the other. The converter genuinely has three output voltage states, so enumeration covers reality and the set size cannot be chosen. Vehicle actuation is continuous, so the 17 by 8 grid is a design parameter trading tracking quality against area. That also explains the cost functions: with three candidates a squared error costs one multiplier, whereas across a large parallel set a squaring multiplier per candidate is expensive, which is why this design uses a clamped L1 norm.

The prediction step is linear in one case and nonlinear in the other. A single multiply-accumulate per candidate lets the reference design fit its whole controller into a 30 ns critical path. The bicycle model needs the sine and cosine of the predicted heading, which is where the quarter-folded tables and the shift-based rescaling originate. Nothing comparable arises in a linear circuit model.

Resource figures are not directly comparable. The reference reports the controller alone, while the figures here cover the complete design including interconnect, reset and register file. Only the DSP count can be attributed cleanly, and its dominant term is the 17 kinematic cones, which have no counterpart in a three-state converter.

The two projects validate different claims. The reference validates behaviour: the logic-analyser waveforms show the controlled current tracking its reference, which is the right bar when the Simulink model is itself the design intent. This project validates identity: the same integers out of MATLAB, out of RTL simulation and out of silicon at every sample, and in the closed loop the same double-precision trajectory to the print resolution. Identity is the right bar here because the MATLAB source is the specification and any divergence would mean the deployed controller is not the one that was tuned and analysed.

8. Reproduction procedure

The steps below reproduce the result from the repository contents. Absolute paths are those used during development and should be adapted. Keep the working directories outside any cloud-synchronised folder; file locking during synthesis is a recurring failure mode, described in Appendix B.

8.1 MATLAB: golden run and artifact generation

Place `fcs_mpc_v4.m`, `fcs_mpc_v4_tb.m` and `fcs_mpc_v4_hdltb.m` from the `matlab` folder in the working directory, then run the testbench.

```
>> stats = fcs_mpc_v4_tb;
Starting Simulation (Speed Profile Mode)...
Total Laps: 2.54 | Max Error: 0.866 m | Top Speed: 10.20 m/s
```

The run writes every downstream artifact from a single source, which is what keeps the implementations from drifting apart:

- nine `.dat` vector files (`x`, `y`, `psi`, `v`, `ref_x`, `ref_y`, `ref_v` and the two expected commands), one 4-digit hexadecimal word per sample;
- `reference_track.h`, containing the 358-vertex path, the VREF speed table, the plant constants and the scale factors, for the C harness;
- `matlab_closed_loop.csv`, the golden trajectory with the per-step integer inputs and outputs, in 15 columns.

Acceptance for this stage: at least 2.5 laps, peak error below 1.0 m, top speed at 10.2 m/s, and a speed trace that follows the profile rather than saturating. Then confirm that the vectors describe the same run:

```
>> fcs_mpc_v4_hdltb
HDL TB vectors: 801 / 801 match
```

This check gates everything after it. If it is not 801 of 801 in plain MATLAB, hardware generation is premature.

8.2 HDL Coder: generating the VHDL

Generate from the command line rather than from the Workflow Advisor. Passing explicit `int16` arguments tells the code generator the types directly, so the fixed-point conversion stage is never invoked. That stage otherwise stalls on branches the trajectory never exercises, such as the overspeed clamp, and it may re-quantise the hand-chosen widths.

```
hdlcfg = coder.config('hdl');
hdlcfg.TargetLanguage      = 'VHDL';
hdlcfg.TestBenchName       = 'fcs_mpc_v4_hdltb';
hdlcfg.GenerateHDLTestbench = true;    % R2021b spelling: lowercase b
hdlcfg.SimulateGeneratedCode = false;
hdlcfg.InputPipeline       = 1;        % registered inputs
hdlcfg.OutputPipeline      = 1;        % registered outputs

codegen -config hdlcfg fcs_mpc_v4 \
  -args {int16(0),int16(0),int16(0),int16(0),int16(0),int16(0),int16(0)}
```

On R2021b the property is spelled `GenerateHDLTestbench` with a lowercase `b`. The spelling used in later documentation does not bind on this release and fails silently, which presents as a missing testbench after an apparently successful run. The output lands in `codegen\fcs_mpc_v4\hdlsrc`: the core, its types package, the generated testbench and its stimulus files.

8.3 Vivado: IP packaging, implementation and export

1. **Project.** Create an RTL project named `mpc_vhdl` with default part `xc7z007s-clg400-1`. Selecting the wrong Zynq part is a common error and produces a bitstream that will not configure the Cora.
2. **Package the IP.** Tools, then Create and Package New IP, then Create a new AXI4 peripheral. Name it `MPC_controller_AXI`, one AXI-Lite slave named `S00_AXI`, 32-bit data, 20 registers. Replace the wizard skeleton with the two `MPC_controller_AXI` files from the `hdl` folder and add `fcs_mpc_v4.vhd` and `fcs_mpc_v4_pkg.vhd`. Update both the VHDL Synthesis and the VHDL Simulation file groups; updating only the first leaves simulation running against a stale core. Compile order is package, core, wrapper, top. Remove the Software Driver file group, whose empty C files break the Vitis build.
3. **Block design.** Create `design_1`, add the ZYNQ7 Processing System and apply the board preset. Register the IP repository under Settings, IP, Repository. Add `MPC_controller_AXI` and run connection automation. In the Address Editor assign the peripheral to `0x43C0_0000` with a 64 kB range, then validate and create an HDL wrapper.
4. **Constraints.** Add `hw/mpc_timing.xdc` to the constraints file set. Use plain constraint syntax only; conditionals and expression evaluation are rejected by the XDC parser.
5. **Synthesis.** Run synthesis with a single job; parallel jobs exhaust memory on a 16 GB machine and fail without a clear diagnosis. Check utilisation before implementing, since an over-utilised design aborts before the placer and produces no timing report.

```
launch_runs synth_1 -jobs 1
wait_on_run synth_1
open_run synth_1
report_utilization -hierarchical
```

6. **Implementation and bitstream.** Run implementation to write `_bitstream`, again with one job, then read the routed reports.

```
launch_runs impl_1 -to_step write_bitstream -jobs 1
wait_on_run impl_1
# reports under mpc_vhdl.runs/impl_1/ :
# design_1_wrapper_timing_summary_routed.rpt    WNS, TNS, WHS, THS
# design_1_wrapper_utilization_placed.rpt       LUT, FF, DSP, BRAM
# design_1_wrapper_route_status.rpt             unrouted nets
```

7. **Export.** File, Export, Export Hardware, with Include bitstream selected. Verify the archive before leaving Vivado; without the bitstream, Vitis silently runs whatever was previously configured in the fabric.

```
unzip -l design_1_wrapper.xsa | grep .bit
```

Close the Hardware Manager before starting Vitis. Vivado holds the JTAG cable and Vitis will report the target as busy.

8.4 Vitis: closed-loop application

1. **Platform.** Create a platform project from the exported XSA, standalone, `ps7_cortexa9_0`, 32-bit, and build it. Create a fresh platform after an IP change rather than updating an existing one, which leaves the metadata inconsistent. Confirm the board support package found the peripheral: `xparameters.h` should define `XPAR_MPC_CONTROLLER_AXI_0_S00_AXI_BASEADDR` as `0x43C00000`.
2. **Application.** Create an empty C application on that platform and copy `mpc_closed_loop.c` and `reference_track.h` from `src` into its `src` folder.
3. **Build settings.** Under C/C++ Build, Settings, add `-ffp-contract=off` to the compiler flags. This is mandatory, not advisory. The FPGA backend selects itself automatically on an ARM target; `-DMPC_BACKEND_FPGA` forces it explicitly. Confirm that the standard output of the standalone domain is the processor UART.
4. **Host check first.** Before touching the board, build the same source on a workstation with `-DMPC_BACKEND_SW` and compare its CSV against the golden. This isolates any error in the C layer from any error in the hardware path, and it takes seconds rather than minutes.

```
gcc -O2 -std=c11 -ffp-contract=off mpc_closed_loop.c -lm -o mpc_cl
./mpc_cl c_closed_loop.csv
backend=SW steps=801 laps=2.54 max_err=0.866 m
```

5. **Run on hardware.** Connect the board over USB, which enumerates as combined JTAG and UART. Open the serial terminal at 115200 8N1 with logging enabled before programming, because output begins as soon as the boot loader hands off and a late terminal loses the header and first rows. Then Run As, Launch on Hardware. Let all 801 rows stream out, stop the log, and rename the capture to `c_closed_loop.csv`.

The footer should read `backend=FPGA steps=801 laps=2.544944 max_err=0.866 m`.

8.5 Comparison

Place `compare_fpga_matlab.m`, `matlab_closed_loop.csv` and `c_closed_loop.csv` in one directory, set it as the MATLAB working directory, and run the script. Both CSV files need at least 15 columns, with the header ending in `err,vref`. A 14-column golden file from an earlier revision makes the reader discard every row.

```
>> compare_fpga_matlab
Closed-loop comparison: c_closed_loop.csv vs matlab_closed_loop.csv (801 steps)
  int-exact rows (state+ref+cmd) : 801 / 801
  first integer mismatch         : none
  accel / steer mismatches       : 0 / 0
  max |position| diff            : 7.070e-07 m
  vref (profile) mismatches      : 0
  MATLAB top speed / C top speed : 10.20 / 10.20 m/s
PASS: command sequences bit-identical; trajectories match.
```

Acceptance for the stage: identical integer columns at every step, position difference at or below the print resolution, and the same lap count on both sides. A real divergence points at either a controller mismatch, which the RTL testbench will localise, or a non-deterministic plant primitive, which usually means a library call has reappeared or the contraction flag was omitted.

9. Planned work

9.1 Configuration A: known circuit

This is the configuration described throughout the report and it remains a target rather than a legacy mode. A surveyed circuit with an offline speed profile is the right formulation whenever the geometry is available in advance, and it produces the best achievable lap time because the profile can look arbitrarily far ahead. Three items remain open.

1. Raise the corner speed limit. The lateral acceleration limit is currently 4 m/s^2 , which is conservative for a passenger vehicle on dry asphalt. Increasing it lifts the whole profile, and the cost weights need to be revisited at the same time so that the overspeed penalty continues to dominate the reward at the new limit.
2. Reduce peak tracking error. Two levers are available and neither requires more logic than the device can supply: scheduling the look-ahead gain with speed, and refining the steering grid near zero, where the present spacing of two units is coarse relative to the corrections actually needed on the straights.
3. Pack the input writes into a burst. Section 6.4 shows that 96.8 percent of the per-step cost is bus traffic and that each of the ten AXI transactions costs about 342 ns because the memory barrier in every write prevents them from overlapping. Replacing the seven separate register writes with one burst, or with a wider write-combining interface, is the only change that would materially reduce end-to-end latency; a faster controller would not.

9.2 Configuration B: unknown circuit

The intended end state supports both configurations from the same hardware. In the unknown-track configuration the vehicle has no surveyed map and no precomputed profile, and the reference is whatever the perception and estimation layer can supply from the road ahead. Two variants are planned, in increasing order of capability.

Variant B1, constant conservative reference speed. The offline profile is removed and the processing system writes a single fixed value to the `REF_V` register at every step, chosen so that the vehicle can negotiate the tightest curvature the road is expected to present. This requires no change to the programmable logic, no change to the register map and no change to the controller, since `REF_V` is already a per-step input rather than a

compiled constant. It costs lap time on open sections, where the vehicle drives well below the available limit, but it is safe against curvature the controller has not seen. It is the natural first step because it can be validated against the existing hardware with a software change alone.

Variant B2, online profile over a receding window. The profile generator of Section 2.4 is reformulated to run over a short window of the path ahead rather than over a closed circuit, and executed each step on the processing system. The curvature estimate and the lateral limit carry over unchanged. The backward braking pass is the part that needs care: over a finite window it must terminate in a conservative boundary condition, typically the speed at which the vehicle could stop within the remaining visible distance, so that the profile never commits to a speed from which it cannot recover if the road tightens beyond the horizon. The forward acceleration pass carries over directly. This variant restores most of the speed advantage of the offline profile without prior knowledge of the circuit, and its accuracy is bounded by how far ahead the path is known rather than by the controller.

Both variants leave the MPC core untouched. That is a direct consequence of the interface decision to expose the reference speed as a register write rather than to compile the profile into the hardware. The work sits in the processing-system software and in the path estimator that feeds it, and the validation methodology of Section 5 transfers unchanged, with the golden model updated to whichever reference generator is under test.

9.3 Engineering backlog

1. Re-measure once the interface changes. The instrumentation is in the harness and costs 43 ns per timestamp, so any interface change can be quantified immediately by re-running the closed loop and reading the footer.
2. Extend the circuit generator with further geometries, including tighter radii and compound curvature, to stress the controller beyond the present layout.
3. Feed live inertial measurements to the processing system and insert a state estimator ahead of the quantisation step. The integrated plant is then replaced by a measured state, which gives a more realistic hardware-in-the-loop run.
4. Re-verify the timing exception. The restructured core may now close under plain single-cycle analysis, which would allow the multicycle constraint to be retired. This should be confirmed by reporting the requirement on the path from the input registers to the output registers before the constraint is removed.

Appendix A: register map

Base address 0x43C0_0000, aperture 64 kB, AXI4-Lite, 32-bit data. All controller payloads are 16-bit values placed in the low half of a 32-bit word. The two command outputs carry signed 6-bit payloads that the processor masks and sign-extends.

Offset	Name	Access	Contents
0x00	CTRL	read/write	Bit 0 kick, self-clearing; bit 1 enable; bit 2 interrupt enable
0x04	STATUS	read only	Bit 0 done; bit 1 busy
0x10	X	read/write	Vehicle x position, int16, 64 units per metre
0x14	Y	read/write	Vehicle y position, int16, 64 units per metre
0x18	PSI	read/write	Heading wrapped to plus or minus pi, int16, 41 units per radian
0x1C	V	read/write	Speed, int16, 64 units per m/s
0x20	REF_X	read/write	Look-ahead reference x, int16, same scale as X
0x24	REF_Y	read/write	Look-ahead reference y, int16, same scale as Y
0x28	REF_V	read/write	Profile speed at the current vertex, int16, same scale as V
0x30	ACCEL	read only	Acceleration command, signed 6-bit payload
0x34	STEER	read only	Steering command, signed 6-bit payload
0x40	TICK	read only	Completed-solve counter, increments on each capture

The write decoder protects the status and result registers, so a stray write cannot forge a command. All seven input registers are readable, which is what makes the interface integrity check of Section 5.3 possible: the processor reads back what the peripheral actually latched rather than trusting what it sent.

Appendix B: pitfalls encountered and their resolutions

The following are failures that occurred during development and cost non-trivial time. They are recorded because most of them present with a symptom that does not point at the cause.

Symptom	Cause and resolution
Implementation aborts before the placer, no timing report produced	Over-utilisation, not a timing failure. Check utilisation after synthesis. A design that does not fit produces no timing report at all.
Synthesis fails with no clear diagnosis on a 16 GB machine	Parallel synthesis jobs exhaust memory. Use <code>-jobs 1</code> .
Constraint file rejected by the parser	XDC accepts a restricted Tcl subset. Conditionals, expression evaluation, loops and printing are not permitted; use plain constraint statements only.
HDL Coder produces no testbench although the option was selected	On R2021b the property is <code>GenerateHDLTestbench</code> with a lowercase <code>b</code> . The alternative spelling does not bind and fails silently.
RTL simulation passes but hardware behaves differently after an IP edit	Only one file group was updated when repackaging. Both VHDL Synthesis and VHDL Simulation must carry the new sources.
Closed-loop run produces 801 rows of zeros and no motion	The reference speed register was never written, so the controller sees zero target speed. Confirm the write to offset <code>0x28</code> and read the register back.
Comparison script reports an empty comparison	Column-count mismatch between the golden CSV and the capture. Both must have 15 columns; the header ends in <code>err,vref</code> .
Trajectories diverge slowly in the closed loop	A library math call reappeared, or <code>-ffp-contract=off</code> was omitted so multiply-add pairs fused. Both break bit-exact agreement of the plant.
Vitis platform not found after an IP change	Updating an existing platform leaves inconsistent metadata. Create a fresh platform from the new XSA.
File-locking errors during synthesis or simulation	The project sits inside a cloud-synchronised folder. Move it to a local path with no spaces.
First latency sample is much larger than the rest	Cold caches, an untrained branch predictor and the first traffic through the interconnect. Expected, not a fault. Identify the first sample rather than discarding it silently.
Captured CSV has no trailing footer line	The terminal log was stopped immediately after the last row. Harmless, since the comparison script recomputes lap count and peak error from the rows themselves.

References

- [1] M. Purraji, H. Zamiri and M. de Castro, 'Easy and Straightforward FPGA Implementation of Model Predictive Control Using HDL Coder', *Electronics*, vol. 14, no. 3, art. 419, 2025. Included in docs/.
- [2] T. Desai, B. Plancher and R. I. Bahar, 'Real-Time, Energy-Efficient, Sampling-Based Optimal Control via FPGA Acceleration', *arXiv:2601.17231*, 2026.
- [3] K. Nguyen, S. Schoedel, A. Alavilli, B. Plancher and Z. Manchester, 'TinyMPC: Model-Predictive Control on Resource-Constrained Microcontrollers', *IEEE Int. Conf. on Robotics and Automation (ICRA)*, Yokohama, 2024.
- [4] I. Mahajan, K. Nguyen, S. Schoedel, E. Nedumaran, M. Mata, B. Plancher and Z. Manchester, 'Code Generation and Conic Constraints for Model-Predictive Control on Microcontrollers with Conic-TinyMPC', *arXiv:2403.18149*, 2024.
- [5] R. Lara-Lopez, I. Gonzalez-Prieto, J. Carrillo-Rios, J. J. Aciego, P. Mora-Moreno, M. J. Duran and A. Gonzalez-Prieto, 'Adaptive Sampling-Time Multivector Model Predictive Control for Six-Phase Induction Motor Drives', *Machines*, vol. 14, no. 6, art. 592, 2026.
- [6] J. P. Allamaa, P. Listov, H. Van der Auweraer, C. Jones and T. D. Son, 'Real-time Nonlinear MPC Strategy with Full Vehicle Validation for Autonomous Driving', *American Control Conference*, 2022, pp. 1982–1987.
- [7] R. Verschueren et al., 'acados: a modular open-source framework for fast embedded optimal control', *Math. Prog. Comp.*, vol. 14, pp. 147–183, 2022.
- [8] Project repository: github.com/subdiduntil2/FPGA-Accelerated-Model-Predictive-Control